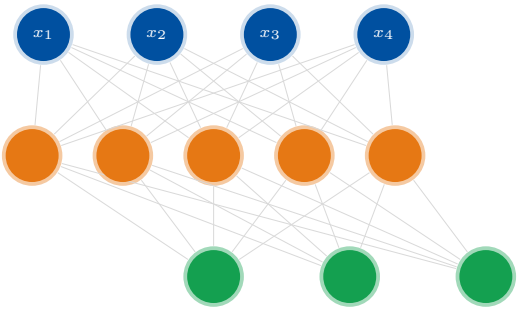


Synapse

Forging Neural Networks in Raw C++

A Complete Guided Textbook

From Zero C++ Knowledge to a Working
Multi-Layer Perceptron — Built from Scratch

```
// No PyTorch.  No TensorFlow.  No Eigen.
   // Just C++, math, and raw memory.

                MLP net(2, {
  LayerConfig{8, Activation::ReLU, 0.0},
  LayerConfig{1, Activation::Sigmoid, 0.0}
                });
          net.fit(X, Y, cfg);
```

Based on the open-source Synapse.cpp project

2026

Contents

Preface

Welcome to **Synapse** — a textbook unlike most you have encountered. This book has exactly one goal: to walk you, a motivated student fresh out of 12th grade, from zero knowledge of `C++` to a complete, working, *zero-library* neural network implemented entirely by hand.

You already know what a neuron is. You have seen the forward pass equations. You understand — at least conceptually — that backpropagation flows gradients backwards through a chain rule. What you *don't* know yet is how any of that becomes actual running code, especially in `C++`, a language famous for demanding that you manage your own memory.

The project we will dissect is called **Synapse.cpp**: five source files, roughly 1 000 lines of pure `C++`17, that implement:

- A self-contained `Matrix` class with matmul, broadcasting, and all the math a neural net needs.
- A full `MLP` class with arbitrary depth, six activation functions, three loss functions, two optimizers (SGD and Adam), dropout, and gradient clipping.
- Four live demo experiments: XOR, Circle, Sine Regression, and 3-Class Gaussian Blobs.

How this book is structured:

Chapter 1 — The `C++` Bare Metal
Everything you need to know about the language before reading a single line of the project: memory, pointers, references, headers, the heap, and operator overloading.

Chapter 2 — The Matrix Engine
A deep dive into `Matrix.hpp` and `Matrix.cpp`: flat-array storage, the Rule of Three, element-wise operators, broadcasting.

Chapter 3 — The Architecture
`MLP.hpp` explained: structs, enums, `std::vector`, and how layers are described in data.

Chapter 4 — The Brain
`MLP.cpp` line by line: forward pass, backpropagation, SGD, Adam.

Every chapter mixes three things: *plain English explanations*, *annotated code snippets*, and *TikZ diagrams* that make the invisible visible. Read them together — don't skip the diagrams.

Let's build a brain.

Chapter 1

The C++ Bare Metal

Before we read a single line of `Synapse.cpp`, we need to understand the language it is written in. This chapter is a crash course in every C++ feature the project uses — taught from the ground up, with pictures.

1.1 What Is C++ and Why Does It Matter Here?

Python, the language you probably learned first, takes care of a huge amount of bookkeeping for you. When you write `x = [1, 2, 3]`, Python silently allocates memory, tracks how many variables point at that list, and frees the memory when you are done. This convenience comes at a cost: speed and control.

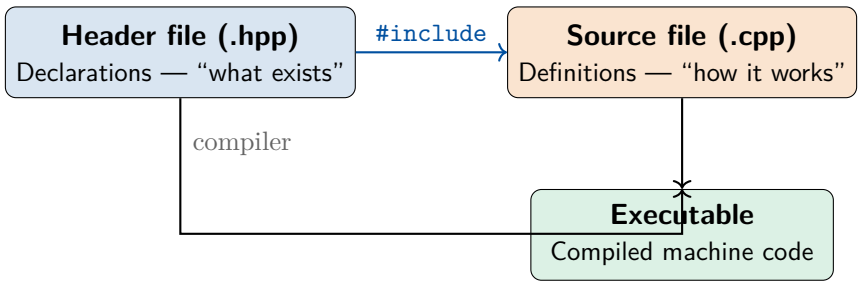
C++ hands you the keys to the car. You decide when memory is allocated, where it lives, and when it is freed. This makes C++ programs dramatically faster — often 10–50× compared to Python for tight numerical loops — and it means you can write a matrix library that performs thousands of matrix multiplications per second without any hidden overhead.

Why zero libraries?

Most ML code uses PyTorch or NumPy. Those libraries are themselves written in C/C++. By writing Synapse.cpp from scratch, we see *exactly* what those libraries do under the hood. There is no magic — only math and memory management.

1.2 Files, Headers, and the `#include` System

A C++ project is split into two kinds of files:



The header file is like a *table of contents* for a book. It tells every other file "here is what I offer". The source file contains the actual chapters. When you write:

```
#include "Matrix.hpp"
```

you are asking the compiler to copy-paste the entire contents of `Matrix.hpp` at that point. It is purely textual substitution — the compiler never sees the `#include` directive; it only sees the expanded result.

1.2.1 Header Guards — Preventing Double Inclusion

What happens if two files both include `Matrix.hpp`? Without protection, the class would be declared twice, causing a compiler error. Header guards prevent this:

```
#pragma once    // Modern equivalent of the old #ifndef guard
                // Tells the compiler: include this file at most once
                // per translation unit.

class Matrix { /* ... */ };
```

Listing 1.1: Header guard in Matrix.hpp

`#pragma once` is supported by every modern compiler (GCC, Clang, MSVC) and is simpler than the old `#ifndef / #define / #endif` pattern.

1.3 Variables, Types, and Memory — The Fundamentals

Every variable in C++ has a *type* that determines exactly how many bytes of memory it occupies.

Type	Bytes	Range (approx.)	Use in Synapse
`int`	4	-2^{31} to $2^{31}-1$	indices, loop counters
`double`	8	$\pm 1.8 \times 10^{308}$, 15 sig. digits	all ML math
`bool`	1	`true` / `false`	training mode flag
`double*`	8	address (pointer to double)	the matrix data buffer

`double` is used for all numerical computation in Synapse.cpp because floating-point precision matters in training — tiny rounding errors accumulate over thousands of gradient steps.

1.4 The Stack and the Heap — Where Variables Live

This is the single most important concept for understanding C++. Every variable lives in one of two memory regions:

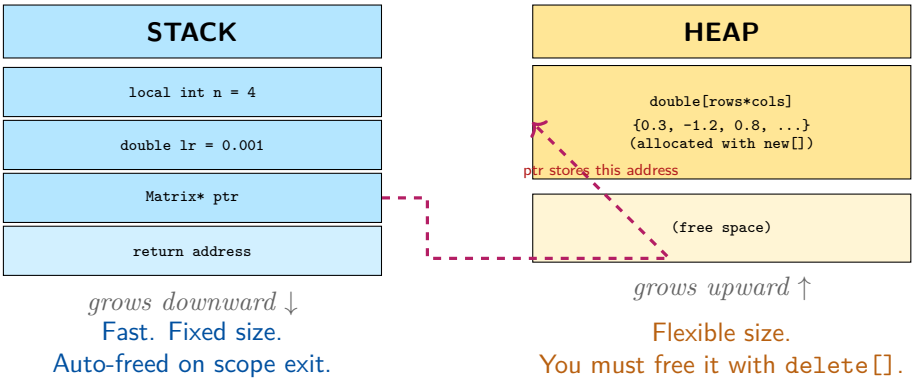


Figure 1.1: Stack vs. Heap memory layout. The stack holds small, fixed-size locals. The heap holds dynamically sized data like our matrix arrays.

- **Stack memory** is fast, automatically managed, and limited in size (≈8 MB typically). Local variables in functions live here. When the function returns, the stack frame is gone — all locals are destroyed.
- **Heap memory** is vast (gigabytes), persists until you explicitly free it, and is accessed through *pointers*. This is where the matrix data arrays live.

1.4.1 The `new` and `delete` Keywords

To allocate on the heap, use `new`:

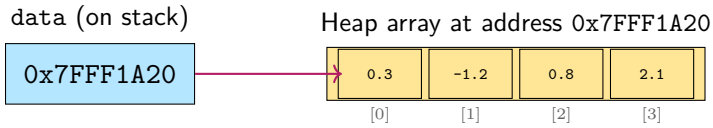
```
// Allocate an array of 12 doubles on the heap:
double* data = new double[12];   // returns a pointer to the first
    element

// Use it:
data[0] = 3.14;
data[5] = -1.0;

// When done, FREE IT:
delete[] data;     // must use delete[] (not delete) for arrays
data = nullptr;    // good practice: set pointer to null after freeing
```

The Golden Rule of Heap Memory

Every `new` must have exactly one matching `delete`. Forgetting `delete` causes a **memory leak** — your program silently consumes RAM until the OS kills it. Calling `delete` twice causes **undefined behaviour** — your program may crash or corrupt data randomly.

1.5 Pointers — The Addresses of Memory

A pointer is a variable that stores a *memory address*. If `data` is a `double*`, then:




```cpp
double* data = new double[4]{0.3, -1.2, 0.8, 2.1};

// Dereferencing: get the VALUE at the address
double first = *data;           // = 0.3  (same as data[0])
double third = *(data + 2);     // = 0.8  (same as data[2])

// Subscript notation (syntactic sugar for pointer arithmetic):
double second = data[1];        // = -1.2
```

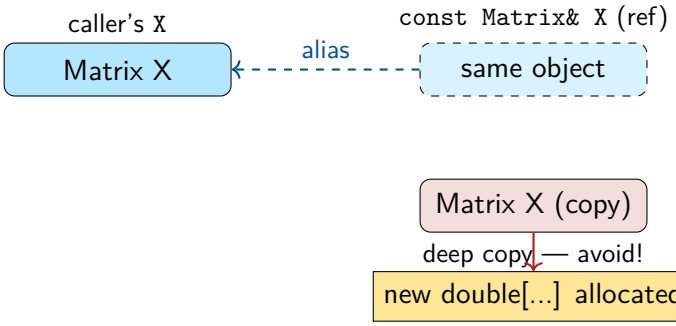
`data[i]` is exactly equivalent to `*(data + i)`. The compiler multiplies `i` by the size of `double` (8 bytes) and adds that offset to the base address.

1.6 References — Aliases Without Copying

A reference is like a permanent alias for another variable. Unlike a pointer, it cannot be null, and you never need to dereference it:

```cpp
void scale(Matrix& m, double factor) {  // m is a reference - no copy!
    for (int i = 0; i < m.rows * m.cols; ++i)
        m.data[i] *= factor;            // modifies the ORIGINAL matrix
}

// Compare with:
void scale_copy(Matrix m, double factor) {  // m is a COPY - expensive!
    // changes here do NOT affect the caller's Matrix
}
```

In Synapse.cpp, functions like `backward(const Matrix& X, ...)` take `const` references — they can read the matrix but not modify it, and no expensive copy is made.



1.7 Functions and Scope

C++ functions look very similar to Python but with type annotations on everything:

```
// Python:          def add(a, b): return a + b
// C++:
double add(double a, double b) { return a + b; }

// Return type ^     ^ parameter types
```

Scope: a variable exists from its declaration to the closing `}` of its enclosing block. When it goes out of scope, its *destructor* runs automatically. For stack objects that *own* heap memory (like our `Matrix` class), this is where cleanup happens.

1.8 Classes and `struct` — Bundling Data and Behaviour

A class bundles data (member variables) and functions (member functions / methods) together:

```
class Matrix {
public:             // public: accessible from anywhere
    int rows, cols;
    double* data;  // pointer to heap array

    Matrix(int r, int c);    // constructor
    ~Matrix();               // destructor (called on scope exit)
    double& at(int r, int c);   // member function
};
```

`struct` is identical to `class` in C++, except members are `public` by default instead of `private`. In Synapse.cpp, `LayerConfig` and `TrainConfig` are `struct`s because they are pure data containers.

1.9 Operator Overloading

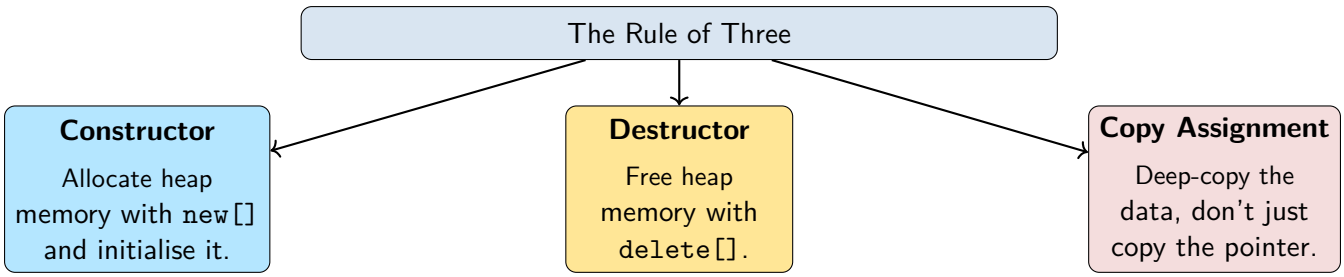
One of C++'s most powerful features: you can define what operators like `+`, `*`, `-` mean for your own types.

```
// Without overloading:
Matrix result = a.add(b);     // verbose and ugly

// With overloading:
Matrix result = a + b;        // reads like math!
```

In Synapse.cpp, the `Matrix` class overloads `+`, `-`, `*`, `/` for both element-wise matrix operations and scalar operations. This is how the backpropagation code reads almost like the mathematical formulas it implements.

1.10 The Rule of Three (Constructor, Destructor, Copy Assignment)

When a class *owns* heap memory through a pointer, you **must** define three special functions — otherwise C++ will generate dangerously incorrect defaults.



Why does the default copy break things? The compiler-generated copy constructor simply copies each member variable. For a pointer, that means two `Matrix` objects end up *sharing the same heap array*. When either is destroyed, `delete[]` runs on that array — leaving the other matrix pointing at freed ("dangling") memory.

```
// Imagine the compiler generates this (BAD):
Matrix::Matrix(const Matrix& other) {
    rows = other.rows;
    cols = other.cols;
    data = other.data;   // DANGER: both objects now share the same
        pointer!
}
// When 'other' is destroyed: delete[] data is called.
// Now our 'data' pointer points to freed memory - crash!
```

Listing 1.2: The dangerous default — shallow copy

```
void Matrix::copyFrom(const Matrix& other) {
    alloc(other.rows, other.cols);                  // allocate fresh
        memory
    std::memcpy(data, other.data,                   // copy all the
        values
                sizeof(double) * rows * cols);
}
Matrix::Matrix(const Matrix& o) { copyFrom(o); }  // copy constructor
```

Listing 1.3: The correct deep copy (what Matrix.cpp actually does)

1.11 Move Semantics — Stealing Instead of Copying

C++11 introduced *move semantics*: instead of copying all the data from a temporary object, we can *steal* its heap pointer.

```
Matrix::Matrix(Matrix&& o) noexcept   // && = rvalue reference
    (temporary)
    : rows(o.rows), cols(o.cols), data(o.data)
{
    o.rows = o.cols = 0;
    o.data = nullptr;    // the old object no longer owns the data
}
```

This is how expressions like `Matrix result = a + b;` work efficiently: `a + b` creates a temporary `Matrix`; instead of deep-copying it into `result`, the compiler moves (steals) the heap pointer. Zero copying overhead.

1.12 `std::vector` — A Resizable Array from the Standard Library

Synapse.cpp uses exactly one standard library container: `std::vector<T>`. A vector is a dynamically resizable array — it manages its own heap memory and grows automatically.

```cpp
#include <vector>

std::vector<int> v;           // empty
v.push_back(42);              // append
v.push_back(17);
int x = v[0];                 // = 42, O(1) access
int n = v.size();             // = 2

// Range-for loop:
for (const auto& elem : v) {
    std::cout << elem << "\n";
}
```

In `MLP.hpp`, `std::vector<Layer> layers_;` holds all the layers of the network. When we add a new layer with `push_back`, the vector handles memory for us.

1.13 Enumerations — Named Constants

```cpp
enum class Activation { ReLU, LeakyReLU, Sigmoid, Tanh, Linear,
    Softmax };
```

`enum class` creates a strongly-typed set of named constants. Instead of passing magic integers like `2` to mean "use sigmoid", we pass `Activation::Sigmoid`. The compiler catches typos and mixing of unrelated enums.

1.14 Lambda Functions — Inline Anonymous Functions

C++11 introduced lambdas: small anonymous functions you can write inline.

```cpp
// In MLP.cpp, applyActivation uses a lambda with Matrix::apply:
return Z.apply([](double x){ return x > 0.0 ? x : 0.0; });
//              ^-- lambda: takes a double, returns max(x, 0)
```

The syntax `[capture](parameters){ body }` creates a callable function object.

1.15 Chapter Summary

What we learned in Chapter 1

- C++ splits code into headers (declarations) and source files (definitions). `#include` is textual substitution.
- Variables live on the **stack** (fast, automatic, limited) or the **heap** (large,

manual, accessed via pointers).

- `new` allocates heap memory; `delete`/`delete[]` frees it. Every `new` must have a `delete`.
- Pointers store addresses. References are aliases — no copy, no null.
- The **Rule of Three**: if your class owns heap memory, define a copy constructor, destructor, and copy-assignment operator.
- Operator overloading makes mathematical code readable.
- `std::vector` is a safe, resizable array. `enum class` gives named constants.

Chapter 2

The Matrix Engine: `Matrix.hpp` and `Matrix.cp`

Every neural network ultimately performs one thing repeatedly: *multiply matrices, add biases, apply a function element-wise.* Before we can think about layers, activations, or backpropagation, we need a solid matrix abstraction. This chapter dissects `Matrix.hpp` and `Matrix.cpp` completely.

2.1 The Core Design Decision: Flat 1-D Storage

A matrix is a 2-D object — rows and columns. But computer memory is a 1-D sequence of bytes. Synapse.cpp stores the matrix in a single flat `double` array in **row-major order**: row 0 first, then row 1, and so on.

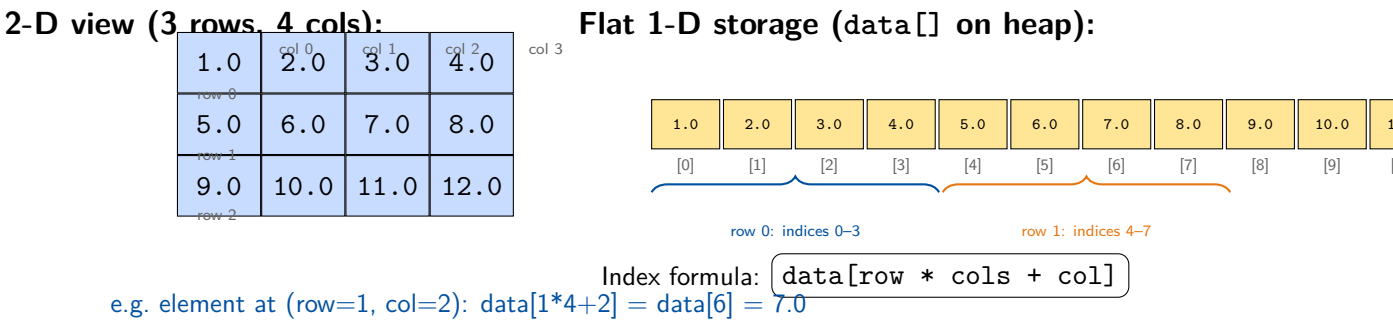


Figure 2.1: Row-major flat storage: the 2-D matrix is stored as a contiguous 1-D array. Element `(r,c)` is at index `r*cols+c`.

This formula **`data[r * cols + c]`** appears hundreds of times in the codebase. Memorise it — it is the key to reading the code.

2.2 The Matrix Class Declaration

Let us read the full class declaration from `Matrix.hpp` section by section.

```
1 class Matrix {
```

```
public:
    int rows, cols;
    double* data;   // flat row-major storage: data[r*cols + c]
```

Listing 2.1: Matrix.hpp — public data members

Three member variables: two integers for dimensions, and a pointer to the heap array. The pointer is declared `public` for performance — in a production library you would make it `private` and add accessors, but for clarity in an educational codebase, direct access is acceptable.

2.2.1 Constructors

```
Matrix();                                  // default: rows=0, cols=0, data=nullptr
Matrix(int rows, int cols, double init = 0.0);  // allocate + fill
Matrix(const Matrix& other);               // copy constructor (deep copy)
Matrix(Matrix&& other) noexcept;           // move constructor (steal pointer)
~Matrix();                                 // destructor (free heap memory)
```

Listing 2.2: Matrix.hpp — constructors

Note the default parameter `double init = 0.0`: if you write `Matrix(3, 4)`, the matrix is zero-initialised. If you write `Matrix(3, 4, 1.0)`, it is filled with 1.0.

2.2.2 Assignment Operators

```
Matrix& operator=(const Matrix& other);      // copy assignment
Matrix& operator=(Matrix&& other) noexcept; // move assignment
```

These return `Matrix&` (a reference to `*this`) so that chained assignments like `a = b = c;` work correctly.

2.3 Implementation: Allocation and Deallocation

```
void Matrix::alloc(int r, int c) {
    rows = r; cols = c;
    data = new double[r * c]();   // () zero-initialises the array
}

void Matrix::dealloc() {
    delete[] data;
    data = nullptr;
    rows = cols = 0;
}

void Matrix::copyFrom(const Matrix& other) {
    alloc(other.rows, other.cols);
```

```
14      std::memcpy(data, other.data, sizeof(double) * rows * cols);
15  }
```

Listing 2.3: Matrix.cpp — private helpers

The () after new double[n]

Writing `new double[n]()` (with parentheses) zero-initialises all elements. Without the parentheses, the values are uninitialised garbage. Always use `()` for safety.

The Full Constructor

```
1  Matrix::Matrix() : rows(0), cols(0), data(nullptr) {}
2
3  Matrix::Matrix(int r, int c, double init) {
4      alloc(r, c);        // allocates r*c doubles on the heap
5      fill(init);         // sets every element to init
6  }
7
8  Matrix::Matrix(const Matrix& o) { copyFrom(o); }    // deep copy
9
10 Matrix::Matrix(Matrix&& o) noexcept                 // move: steal
       pointer
11     : rows(o.rows), cols(o.cols), data(o.data) {
12     o.rows = o.cols = 0;
13     o.data = nullptr;    // old object no longer owns the data
14 }
15
16 Matrix::~Matrix() { dealloc(); }    // free heap memory
```

Listing 2.4: Matrix.cpp — constructors and destructor

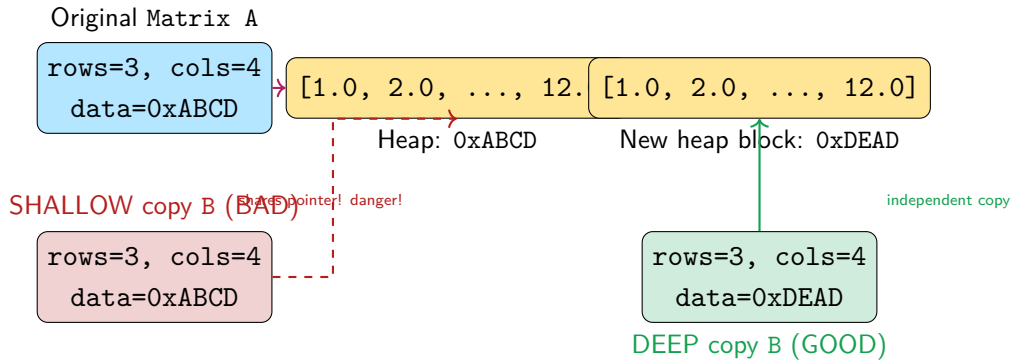


Figure 2.2: Shallow copy shares the pointer (dangerous). Deep copy allocates new memory and copies the values (correct).

2.4 Element Access

```cpp
double& Matrix::at(int r, int c) {
    if (r < 0 || r >= rows || c < 0 || c >= cols)
        throw std::out_of_range("Matrix::at index out of range");
    return data[r * cols + c];   // return a REFERENCE: allows
       m.at(0,0) = 5.0;
}
```

Listing 2.5: Matrix.cpp — element access with bounds checking

Returning a `double&` (reference) means `at()` can be used on both sides of an assignment:

```cpp
M.at(1, 2) = 7.0;        // sets element (1,2) to 7.0
double v = M.at(1, 2);   // reads element (1,2)
```

2.5 Arithmetic Operators — The Heart of the Math

2.5.1 Element-wise Operations

The project uses a clever macro to avoid repeating the same loop four times:

```cpp
#define MATRIX_BINARY_OP(OP)                                    \
    if (rows != b.rows || cols != b.cols)                       \
        throw std::invalid_argument("Matrix shape mismatch"); \
    Matrix out(rows, cols);                                     \
    for (int i = 0; i < rows * cols; ++i)                       \
        out.data[i] = data[i] OP b.data[i];                    \
    return out;

Matrix Matrix::operator+(const Matrix& b) const {
    MATRIX_BINARY_OP(+) }
Matrix Matrix::operator-(const Matrix& b) const {
    MATRIX_BINARY_OP(-) }
Matrix Matrix::operator*(const Matrix& b) const {
    MATRIX_BINARY_OP(*) } // Hadamard!
Matrix Matrix::operator/(const Matrix& b) const {
    MATRIX_BINARY_OP(/) }
```

Listing 2.6: Matrix.cpp — element-wise arithmetic macro

Element-wise vs. Matrix Multiplication

The `*` operator is the **Hadamard (element-wise) product**: `A * B` multiplies corresponding elements. For true matrix multiplication, use `A.dot(B)`. This distinction is critical in backpropagation where both operations appear.

2.5.2 Scalar Operations

```cpp
Matrix Matrix::operator*(double s) const {
    Matrix out(rows, cols);
    for (int i = 0; i < rows*cols; ++i) out.data[i] = data[i] * s;
    return out;
```

```
}
// Also defined: +s, -s, /s, and the in-place versions +=, -=, *=, /=

// Non-member: allows 2.0 * M as well as M * 2.0
Matrix operator*(double s, const Matrix& m) { return m * s; }
```

Listing 2.7: Matrix.cpp — scalar arithmetic

2.5.3 Matrix Multiplication: `dot()`

This is the workhorse of the forward pass. For two matrices A of shape (m, k) and B of shape (k, n), the dot product $C = A \cdot B$ has shape (m, n) with:

$$C_{ij} = \sum_{l=0}^{k-1} A_{il} \cdot B_{lj}$$

```
Matrix Matrix::dot(const Matrix& b) const {
    if (cols != b.rows)
        throw std::invalid_argument(
            "dot: incompatible shapes " + shape() + " x " +
               b.shape());
    Matrix out(rows, b.cols, 0.0);
    for (int i = 0; i < rows; ++i)
        for (int k = 0; k < cols; ++k) {       // loop order: i,k,j
            double aik = data[i * cols + k];    // cache for speed
            for (int j = 0; j < b.cols; ++j)
                out.data[i * b.cols + j] += aik * b.data[k * b.cols
                   + j];
        }
    return out;
}
```

Listing 2.8: Matrix.cpp — matrix multiplication

Loop Order i-k-j for Cache Efficiency

The inner loop iterates over `j`, accessing `out.data[i*b.cols+j]` sequentially (great for cache). The cached `aik` avoids reloading `A[i,k]` inside the innermost loop. This is a classical optimisation for matmul.

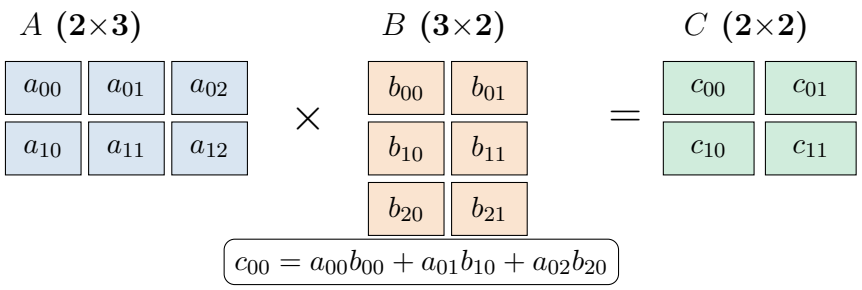


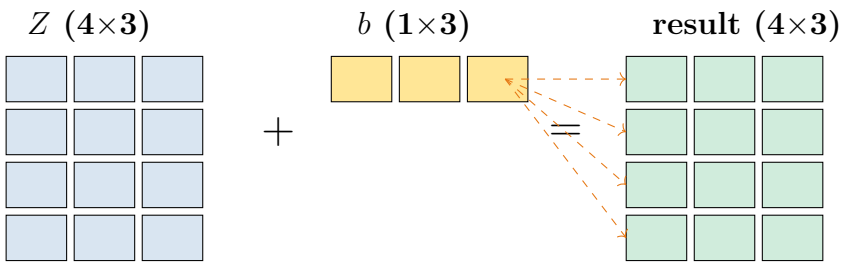
Figure 2.3: Matrix multiplication: each element of C is a dot product of a row of A with a column of B.

2.6 Broadcasting: broadcastAdd

In neural network math, we constantly add a bias vector (shape $1 \times n$) to every row of a result matrix (shape batch $\times n$). This is *broadcasting*.

```
Matrix Matrix::broadcastAdd(const Matrix& rhs) const {
    Matrix out(rows, cols);
    if (rhs.rows == 1 && rhs.cols == cols) {
        // Add a row-vector (1,cols) to every row:
        for (int r = 0; r < rows; ++r)
            for (int c = 0; c < cols; ++c)
                out.data[r*cols+c] = data[r*cols+c] + rhs.data[c];
    } else if (rhs.cols == 1 && rhs.rows == rows) {
        // Add a col-vector (rows,1) to every col:
        for (int r = 0; r < rows; ++r)
            for (int c = 0; c < cols; ++c)
                out.data[r*cols+c] = data[r*cols+c] + rhs.data[r];
    } else {
        throw std::invalid_argument("broadcastAdd: incompatible
           shapes");
    }
    return out;
}
```

Listing 2.9: Matrix.cpp — broadcastAdd



bias row broadcast to all 4 data rows

2.7 Reductions: sum, mean, sum(axis)

```
double Matrix::sum() const {
    double s = 0.0;
    for (int i = 0; i < rows*cols; ++i) s += data[i];
    return s;
}

// Sum along axis 0 (collapse rows → produce (1, cols)):
Matrix Matrix::sum(int axis) const {
    if (axis == 0) {
        Matrix out(1, cols, 0.0);
        for (int r = 0; r < rows; ++r)
            for (int c = 0; c < cols; ++c)
                out.data[c] += data[r * cols + c];
```

```
14          return out;
15      } else {    // axis == 1: collapse cols → (rows, 1)
16          Matrix out(rows, 1, 0.0);
17          for (int r = 0; r < rows; ++r)
18              for (int c = 0; c < cols; ++c)
19                  out.data[r] += data[r * cols + c];
20          return out;
21      }
22  }
```

Listing 2.10: Matrix.cpp — reductions

The axis=0 reduction is used in backpropagation to compute bias gradients: $\frac{\partial L}{\partial b} = \sum_{\text{batch}} \frac{\partial L}{\partial Z}$.

2.8 Element-wise Math: apply, exp, sqrt, square

```
1  // Generic apply: pass any function double→double
2  Matrix Matrix::apply(double (*func)(double)) const {
3      Matrix out(rows, cols);
4      for (int i = 0; i < rows*cols; ++i) out.data[i] = func(data[i]);
5      return out;
6  }
7
8  // Built-in variants:
9  Matrix Matrix::exp()    const { return apply(std::exp); }
10 Matrix Matrix::log()    const { return apply(std::log); }
11 Matrix Matrix::sqrt()   const { return apply(std::sqrt); }
12 Matrix Matrix::square() const {
13     Matrix out(rows, cols);
14     for (int i = 0; i < rows*cols; ++i) out.data[i] = data[i] *
           data[i];
15     return out;
16 }
```

Listing 2.11: Matrix.cpp — apply and derived functions

Function Pointers

`double (*func)(double)` is the type "a pointer to a function that takes one `double` and returns a `double`". Passing `std::exp` passes the address of the standard-library exponential function. The lambda in `applyActivation` (e.g., `[](double x){return x>0?x:0;}`) is compatible with this.

2.9 Weight Initialisation

Before training, weights must be initialised carefully. The project implements four strategies.

```
1  // Xavier Uniform: U[ -sqrt(6/(fanIn+fanOut)),
       +sqrt(6/(fanIn+fanOut)) ]
```

```cpp
void Matrix::xavierUniform(int fanIn, int fanOut) {
    double limit = std::sqrt(6.0 / (fanIn + fanOut));
    randomUniform(-limit, limit);
}

// He Normal: N(0, sqrt(2/fanIn))  - designed for ReLU
void Matrix::heNormal(int fanIn) {
    randomNormal(0.0, std::sqrt(2.0 / fanIn));
}

// Box-Muller transform for normally distributed random numbers:
static double boxMuller() {
    static bool hasSpare = false;
    static double spare;
    if (hasSpare) { hasSpare = false; return spare; }
    double u, v, s;
    do {
        u = ((double)std::rand() / RAND_MAX) * 2.0 - 1.0;
        v = ((double)std::rand() / RAND_MAX) * 2.0 - 1.0;
        s = u*u + v*v;
    } while (s >= 1.0 || s == 0.0);
    double mul = std::sqrt(-2.0 * std::log(s) / s);
    spare = v * mul;
    hasSpare = true;
    return u * mul;
}
```

Listing 2.12: Matrix.cpp — weight initialisation

Why Xavier and He Initialisation?

If weights are too large, activations explode. If too small, gradients vanish. Xavier initialisation keeps the variance of outputs approximately equal to the variance of inputs for sigmoid/tanh layers. He initialisation accounts for the half-zeroing effect of ReLU ($\mathbb{E}[\text{ReLU}(x)^2] = \frac{1}{2}\mathbb{E}[x^2]$), scaling the variance by 2 to compensate:

$$\sigma^2_{\text{He}} = \frac{2}{n_{\text{in}}} \qquad \sigma^2_{\text{Xavier}} = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

2.10 The Transpose

```cpp
Matrix Matrix::transpose() const {
    Matrix out(cols, rows);   // note: swapped dimensions
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c)
            out.data[c * rows + r] = data[r * cols + c];
    return out;
}
```

Listing 2.13: Matrix.cpp — transpose

Transposition turns an $(m \times n)$ matrix into an $(n \times m)$ matrix. In backpropagation, we compute $\frac{\partial L}{\partial W^l} = (A^{l-1})^\top \cdot \delta^l$, which requires transposing the previous layer's activation.

2.11 Chapter Summary

What we learned in Chapter 2

- Matrices are stored flat: `data[r*cols+c]` maps 2-D to 1-D.
- The Rule of Three ensures correct memory management: copy constructor deep-copies, destructor frees.
- `operator*(Matrix)` is element-wise (Hadamard); `dot()` is true matrix multiplication.
- `broadcastAdd` adds a bias row-vector to every row — fundamental to the linear layer.
- `apply(func)` applies any element-wise function uniformly.
- Xavier and He initialisation keep gradients stable from the start.

Chapter 3

The Architecture: `MLP.hpp`

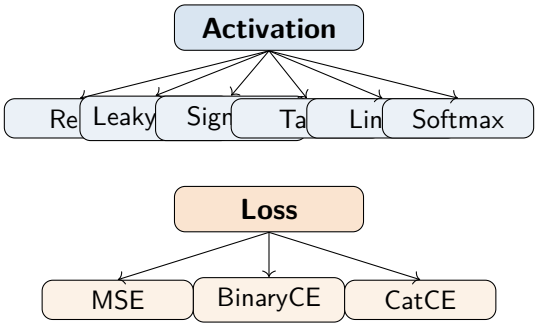
With the matrix engine in place, we can now describe the neural network's *architecture* — the data structures that define what a network *is* before it starts learning.

3.1 Enumerations: Naming the Choices

```
enum class Activation { ReLU, LeakyReLU, Sigmoid, Tanh, Linear,
    Softmax };
enum class Loss       { MSE, BinaryCrossEntropy,
    CategoricalCrossEntropy };
enum class Optimizer  { SGD, Adam };
enum class WeightInit { Xavier, He, RandomUniform, RandomNormal };
```

Listing 3.1: MLP.hpp — enumerations

Each `enum class` groups related named constants into a type. The `class` keyword makes them *scoped* — you must write `Activation::ReLU`, not just `ReLU`. This prevents accidental confusion between, say, `Loss::MSE` and an integer 0.



3.2 LayerConfig — Describing a Single Layer

```
struct LayerConfig {
    int         units;                          // number of neurons
    Activation activation = Activation::ReLU;   // default: ReLU
    double      dropout    = 0.0;               // dropout probability
};
```

Listing 3.2: MLP.hpp — LayerConfig struct

A `LayerConfig` is pure data — three fields that describe what a layer should look like. No methods, no behaviour. The user builds a network by constructing a `std::vector<LayerConfig>`:

```
// Example: 2-input → 8 hidden (ReLU) → 1 output (Sigmoid)
std::vector<LayerConfig> topology = {
    LayerConfig{8,  Activation::ReLU,    0.0},
    LayerConfig{1,  Activation::Sigmoid, 0.0}
};
```

3.3 TrainConfig — All Hyperparameters in One Place

```
struct TrainConfig {
    int        epochs         = 100;
    int        batchSize      = 32;
    double     learningRate   = 0.001;
    Loss       loss           = Loss::MSE;
    Optimizer  optimizer      = Optimizer::Adam;
    WeightInit weightInit     = WeightInit::Xavier;
    double     momentum       = 0.9;    // SGD momentum / Adam beta1
    double     beta2          = 0.999;  // Adam beta2
    double     epsilon        = 1e-8;   // Adam epsilon
    double     gradClip       = 0.0;    // 0 = disabled
    bool       verbose        = true;
    int        printEvery     = 10;
};
```

Listing 3.3: MLP.hpp — TrainConfig struct

Using a `struct` with default values means callers only need to change what differs from the defaults:

```
TrainConfig cfg;            // all defaults
cfg.epochs = 500;           // override just what we need
cfg.learningRate = 0.01;
cfg.loss = Loss::BinaryCrossEntropy;
```

This is the *named parameter idiom* — much cleaner than passing 12 arguments to a function.

3.4 The Layer Struct — Runtime State of One Layer

```
struct Layer {
    // Parameters (what the network learns):
    Matrix W;        // weight matrix  (inFeatures × units)
    Matrix b;        // bias vector    (1 × units)

    // Gradients (computed during backprop):
    Matrix dW;
    Matrix db;
```

```
9
10      // Adam moment estimates:
11      Matrix mW, vW;  // 1st & 2nd moment for W
12      Matrix mb, vb;  // 1st & 2nd moment for b
13
14      // SGD momentum velocity:
15      Matrix velW, velb;
16
17      // Cached forward-pass values (needed for backprop):
18      Matrix Z;         // pre-activation  (batch × units)
19      Matrix A;         // post-activation (batch × units)
20      Matrix dropMask; // dropout mask    (batch × units)
21
22      // Config:
23      Activation activation;
24      double     dropout;
25      int        inFeatures;
26      int        units;
27  };
```

Listing 3.4: MLP.hpp — Layer struct (the runtime object)

A `Layer` holds *everything* associated with one dense layer — parameters, their gradients, optimizer states, and the cached forward-pass values needed for backpropagation. This bundling makes the backward pass clean: given a `Layer&`, we have everything we need.

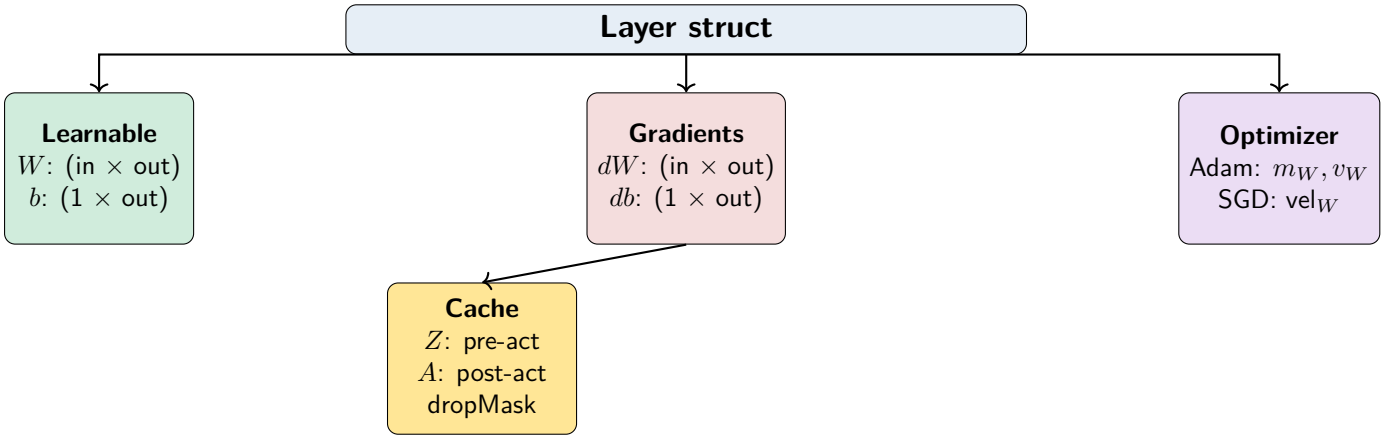


Figure 3.1: The `Layer` struct contains all data associated with one layer, grouped logically.

3.5 The MLP Class Declaration

```
1  class MLP {
2  public:
3      MLP(int inputSize, const std::vector<LayerConfig>& topology);
4
5      Matrix forward (const Matrix& X);
6      double computeLoss(const Matrix& predictions,
7                         const Matrix& targets, Loss lossType);
8      void   backward(const Matrix& X, const Matrix& targets,
```

```
9                       Loss lossType, double gradClip = 0.0);
10      void    updateSGD (double lr, double momentum);
11      void    updateAdam(double lr, double beta1, double beta2, double
            eps);
12      void    fit(const Matrix& X, const Matrix& Y, const TrainConfig&
            cfg);
13
14      Matrix predict(const Matrix& X);
15      void    setTraining(bool training);
16      double accuracy(const Matrix& X, const Matrix& Y);
17      double mse      (const Matrix& X, const Matrix& Y);
18      void    printSummary() const;
19      std::vector<double> getLossHistory() const { return
            lossHistory_; }
20
21  private:
22      int                 inputSize_;
23      std::vector<Layer>  layers_;     // the actual layers (vector
            manages memory)
24      bool                isTraining_ = true;
25      int                 adamStep_   = 0;
26      std::vector<double> lossHistory_;
27
28      Matrix applyActivation (const Matrix& Z, Activation act) const;
29      Matrix activationGrad  (const Matrix& Z, const Matrix& A,
            Activation act) const;
30      Matrix softmax(const Matrix& Z) const;
31      void    initWeights(Layer& layer, WeightInit init);
32  };
```

Listing 3.5: MLP.hpp — the MLP class

The trailing underscore convention

Member variables in `Synapse.cpp` end with an underscore (`inputSize_`, `layers_`). This is a widely-used convention to distinguish member variables from local variables and parameters. It prevents bugs like accidentally shadowing a member with a local of the same name.

3.6 Chapter Summary

What we learned in Chapter 3

- `enum class` creates scoped, strongly-typed named constants — cleaner than raw integers.
- `LayerConfig` is a pure data struct describing the shape and activation of one layer.
- `TrainConfig` bundles all hyperparameters with sensible defaults.
- `Layer` is the runtime struct holding parameters, gradients, optimizer state,

and cached activations.

- MLP aggregates layers in a std::vector<Layer> and exposes fit/predict/forward/backward.

Chapter 4

The Brain: `MLP.cpp`

This is the heart of the project — the implementation of the neural network itself. We will work through every major function: the constructor, the forward pass, activation functions, the loss, backpropagation, and the two optimizers.

4.1 The Constructor: Building the Network

```
1  MLP::MLP(int inputSize, const std::vector<LayerConfig>& topology)
2      : inputSize_(inputSize)   // initialiser list: sets inputSize_
           before body
3  {
4      if (topology.empty())
5          throw std::invalid_argument("MLP: topology must have at
               least one layer");
6
7      int inFeat = inputSize;   // tracks input size as we add layers
8
9      for (const auto& cfg : topology) {
10         Layer layer;
11         layer.units      = cfg.units;
12         layer.activation = cfg.activation;
13         layer.dropout    = cfg.dropout;
14         layer.inFeatures = inFeat;
15
16         // Allocate weight and gradient matrices (proper init
               happens in fit())
17         layer.W  = Matrix(inFeat, cfg.units);   // (inFeat × units)
18         layer.b  = Matrix(1,      cfg.units, 0.0);
19         layer.dW = Matrix(inFeat, cfg.units, 0.0);
20         layer.db = Matrix(1,      cfg.units, 0.0);
21
22         // Adam moments - all start at zero
23         layer.mW  = Matrix(inFeat, cfg.units, 0.0);
24         layer.vW  = Matrix(inFeat, cfg.units, 0.0);
25         layer.mb  = Matrix(1,      cfg.units, 0.0);
26         layer.vb  = Matrix(1,      cfg.units, 0.0);
27
28         // SGD velocity - starts at zero
29         layer.velW = Matrix(inFeat, cfg.units, 0.0);
```

```
30          layer.velb = Matrix(1,       cfg.units, 0.0);
31
32          layers_.push_back(std::move(layer));  // move into the vector
33          inFeat = cfg.units;  // this layer's output is next layer's
                input
34      }
35  }
```

Listing 4.1: MLP.cpp — constructor

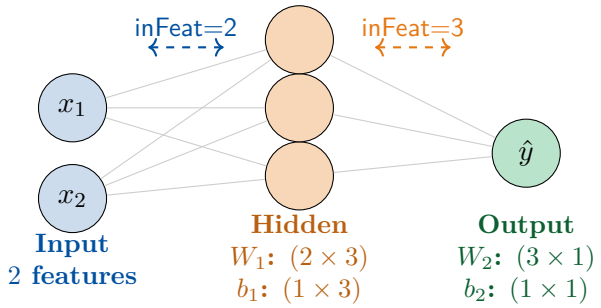


Figure 4.1: The constructor iterates through `LayerConfigs`, updating `inFeat` at each step so each layer's weight matrix is correctly shaped.

4.2 Weight Initialisation

```
1  void MLP::initWeights(Layer& layer, WeightInit init) {
2      switch (init) {
3          case WeightInit::Xavier:
4              layer.W.xavierUniform(layer.inFeatures, layer.units);
                   break;
5          case WeightInit::He:
6              layer.W.heNormal(layer.inFeatures); break;
7          case WeightInit::RandomUniform:
8              layer.W.randomUniform(-0.5, 0.5); break;
9          case WeightInit::RandomNormal:
10             layer.W.randomNormal(0.0, 0.01); break;
11     }
12     layer.b.fill(0.0);   // biases always start at zero
13 }
```

Listing 4.2: MLP.cpp — initWeights

Note that biases are always zero-initialised. This breaks symmetry for the output layer while letting weights be the source of randomness that different neurons learn different features.

4.3 Activation Functions

4.3.1 Implementing the Six Activations

```cpp
Matrix MLP::applyActivation(const Matrix& Z, Activation act) const {
    switch (act) {
        case Activation::ReLU:
            return Z.apply([](double x){ return x > 0.0 ? x : 0.0;
                });
        case Activation::LeakyReLU:
            return Z.apply([](double x){ return x > 0.0 ? x : 0.01 *
                x; });
        case Activation::Sigmoid:
            return Z.apply([](double x){ return
                1.0/(1.0+std::exp(-x)); });
        case Activation::Tanh:
            return Z.apply([](double x){ return std::tanh(x); });
        case Activation::Linear:
            return Z;           // identity - no transformation
        case Activation::Softmax:
            return softmax(Z); // needs special treatment (see below)
    }
    return Z;
}
```

Listing 4.3: MLP.cpp — applyActivation

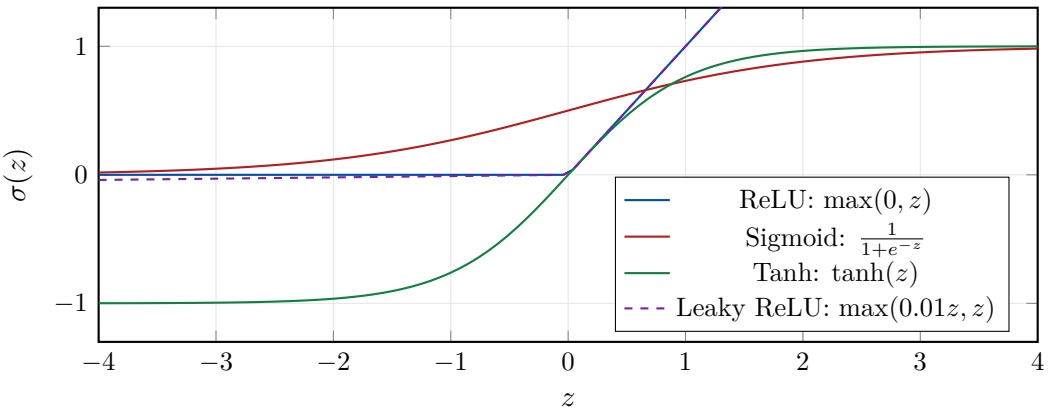


Figure 4.2: The four element-wise activation functions implemented in `Synapse.cpp`. Softmax operates on whole rows and is not shown here.

4.3.2 Numerically Stable Softmax

Softmax cannot be applied element-wise because it requires the full row:

$$\text{softmax}(\mathbf{z})_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

Naïvely computing e^{z_j} overflows to infinity when z_j is large (e.g., 1000). The trick: subtract the row maximum first.

```cpp
Matrix MLP::softmax(const Matrix& Z) const {
    Matrix out(Z.rows, Z.cols);
    for (int r = 0; r < Z.rows; ++r) {
        // Step 1: find the max of this row
```

```
5           double rowMax = Z.data[r * Z.cols];
6           for (int c = 1; c < Z.cols; ++c)
7               if (Z.data[r*Z.cols+c] > rowMax) rowMax =
                    Z.data[r*Z.cols+c];
8
9           // Step 2: compute exp(z - rowMax) and their sum
10          double rowSum = 0.0;
11          for (int c = 0; c < Z.cols; ++c) {
12              out.data[r*Z.cols+c] = std::exp(Z.data[r*Z.cols+c] -
                    rowMax);
13              rowSum += out.data[r*Z.cols+c];
14          }
15
16          // Step 3: normalise
17          for (int c = 0; c < Z.cols; ++c)
18              out.data[r*Z.cols+c] /= rowSum;
19      }
20      return out;
21  }
```

Listing 4.4: MLP.cpp — numerically stable softmax

Why subtracting the max is safe

Note that:

$$\frac{e^{z_j}}{\sum_k e^{z_k}} = \frac{e^{z_j - M}}{\sum_k e^{z_k - M}} \quad \text{where } M = \max_k z_k$$

The M cancels in numerator and denominator, so the result is identical. But now the largest exponent is $e^0 = 1$, and all others are ≤ 1 — no overflow possible.

4.3.3 Activation Gradients (for Backpropagation)

```
1  Matrix MLP::activationGrad(const Matrix& Z,const Matrix&
       A,Activation act) const {
2      switch (act) {
3          case Activation::ReLU:
4              // ReLU'(z) = 1 if z > 0, else 0
5              return Z.apply([](double x){ return x > 0.0 ? 1.0 : 0.0;
                   });
6
7          case Activation::LeakyReLU:
8              return Z.apply([](double x){ return x > 0.0 ? 1.0 :
                   0.01; });
9
10         case Activation::Sigmoid:
11             // sigma'(z) = sigma(z) * (1 - sigma(z)) = A * (1 - A)
12             return A * (A * (-1.0) + 1.0);
13
14         case Activation::Tanh:
15             // tanh'(z) = 1 - tanh^2(z) = 1 - A^2
16             return A.square() * (-1.0) + 1.0;
17
18         case Activation::Linear:
```

```
19          case Activation::Softmax:
20              // Softmax gradient absorbed into loss gradient - return
                    1s
21              return Matrix(Z.rows, Z.cols, 1.0);
22      }
23      return Matrix(Z.rows, Z.cols, 1.0);
24  }
```

Listing 4.5: MLP.cpp — activationGrad

Activation	Forward $\sigma(z)$	Gradient $\sigma'(z)$
ReLU	$\max(0, z)$	$\mathbf{1}[z > 0]$
Leaky ReLU	$\max(0.01z, z)$	0.01 if $z \leq 0$, else 1
Sigmoid	$\frac{1}{1+e^{-z}}$	$A(1 - A)$
Tanh	$\tanh(z)$	$1 - A^2$
Linear	z	1
Softmax	$\frac{e^{z_j}}{\sum e^{z_k}}$	absorbed into CCE gradient

4.4 The Forward Pass

This is where the actual computation happens. Given input X of shape (batch $\times$ inputSize), we compute layer by layer until we reach the output.

```
1  Matrix MLP::forward(const Matrix& X) {
2      Matrix current = X;    // start with raw input
3
4      for (auto& layer : layers_) {
5          // 1. Linear transform: Z = current * W + b
6          layer.Z = current.dot(layer.W).broadcastAdd(layer.b);
7
8          // 2. Activation: A = sigma(Z)
9          layer.A = applyActivation(layer.Z, layer.activation);
10
11         // 3. Dropout (only during training)
12         if (isTraining_ && layer.dropout > 0.0) {
13             layer.dropMask = Matrix(layer.A.rows, layer.A.cols);
14             double keep = 1.0 - layer.dropout;
15             for (int i = 0; i < layer.dropMask.rows *
                    layer.dropMask.cols; ++i) {
16                 double r = (double)std::rand() / RAND_MAX;
17                 layer.dropMask.data[i] = (r < keep) ? (1.0 / keep) :
                        0.0;
18             }
19             layer.A = layer.A * layer.dropMask;
20         } else {
21             layer.dropMask = Matrix(layer.A.rows, layer.A.cols,
                    1.0); // all ones
22         }
23
24         current = layer.A;    // this layer's output is next layer's
               input
```

```
25      }
26
27      return current;  // final layer's output
28  }
```

Listing 4.6: MLP.cpp — forward pass

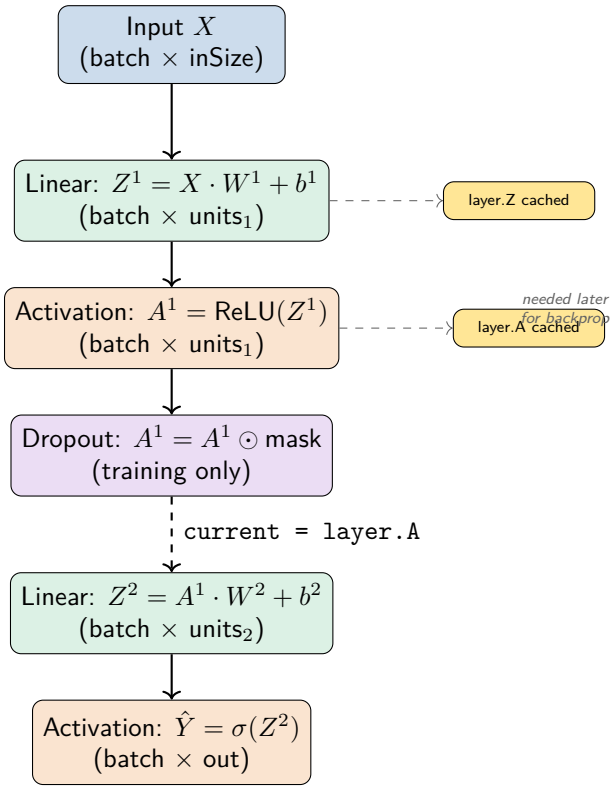


Figure 4.3: The forward pass iterates through layers, storing Z and A at each step for use during backpropagation.

4.4.1 Dropout

Dropout randomly zeroes some activations during training with probability p (the dropout rate). To ensure the expected output is unchanged, the surviving activations are scaled by $\frac{1}{1-p}$ (the "inverted dropout" technique):

```
double keep = 1.0 - layer.dropout;
layer.dropMask.data[i] = (r < keep) ? (1.0 / keep) : 0.0;
//                        ^-- keep this neuron    ^-- kill it
//                         scaled up by 1/keep to maintain
    expectation
```

During evaluation (inference), the dropout mask is all-ones — no neurons are dropped.

4.5 The Loss Functions

```cpp
double MLP::computeLoss(const Matrix& preds,
                        const Matrix& targets, Loss lossType) {
    int n = preds.rows;    // batch size
    switch (lossType) {

        case Loss::MSE: {
            Matrix diff = preds - targets;
            return diff.square().sum() / (2.0 * n);  //
                (1/2n)||y-hat - y||^2
        }

        case Loss::BinaryCrossEntropy: {
            double eps = 1e-12;  // prevent log(0)
            double loss = 0.0;
            for (int i = 0; i < preds.rows * preds.cols; ++i) {
                double p = std::max(eps, std::min(1.0-eps,
                    preds.data[i]));
                double y = targets.data[i];
                loss += -(y * std::log(p) + (1.0-y) *
                    std::log(1.0-p));
            }
            return loss / n;
        }

        case Loss::CategoricalCrossEntropy: {
            double eps = 1e-12;
            double loss = 0.0;
            for (int i = 0; i < preds.rows * preds.cols; ++i) {
                double p = std::max(eps, preds.data[i]);
                loss += -targets.data[i] * std::log(p);
            }
            return loss / n;    // -(1/n) sum_i sum_j y_ij * log(p_ij)
        }
    }
    return 0.0;
}
```

Listing 4.7: MLP.cpp — computeLoss

The Three Loss Functions

MSE (Mean Squared Error) — regression:

$$L = \frac{1}{2n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

Binary Cross-Entropy — binary classification:

$$L = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{p}_i) + (1-y_i)\log(1-\hat{p}_i)\right]$$

Categorical Cross-Entropy — multi-class classification:

$$L = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C} y_{ij}\log(\hat{p}_{ij})$$

The clipping to $[10^{-12}, 1 - 10^{-12}]$ prevents $\log(0) = -\infty$.

4.6 Backpropagation — The Full Implementation

This is the most mathematically dense part of the codebase. We will go through it step by step with both code and diagrams.

4.6.1 The Big Picture

Backpropagation computes $\frac{\partial L}{\partial W^l}$ and $\frac{\partial L}{\partial b^l}$ for every layer l, using the chain rule. The key quantities are:

$$\begin{aligned}
\delta^l &= \frac{\partial L}{\partial Z^l} \quad \text{(the "error signal" at layer } l\text{)} \\
\frac{\partial L}{\partial W^l} &= (A^{l-1})^\top \cdot \delta^l \\
\frac{\partial L}{\partial b^l} &= \sum_{\text{batch}} \delta^l \quad \text{(sum over batch dimension)} \\
\delta^{l-1} &= \delta^l \cdot (W^l)^\top \odot \sigma'(Z^{l-1}) \quad \text{(propagate backward)}
\end{aligned}$$

```
void MLP::backward(const Matrix& X, const Matrix& targets,
                   Loss lossType, double gradClip)
{
    int n = targets.rows;  // batch size
    const Matrix& outputA = layers_.back().A;  // final predictions

    // Step 1: Compute dL/dA for the output layer
    Matrix dA(outputA.rows, outputA.cols);

    switch (lossType) {
        case Loss::MSE:
            dA = (outputA - targets) / (double)n;    // (A-Y)/n
            break;
        case Loss::BinaryCrossEntropy:
            // Combined sigmoid+BCE gradient = (A-Y)/n
            dA = (outputA - targets) / (double)n;
            break;
        case Loss::CategoricalCrossEntropy:
            // Combined softmax+CCE gradient = (A-Y)/n
            dA = (outputA - targets) / (double)n;
            break;
    }

    // Step 2: Backpropagate through layers in reverse
    for (int l = (int)layers_.size() - 1; l >= 0; --l) {
        Layer& layer = layers_[l];

        // Apply dropout mask gradient (zero gradient where neuron
           was dropped)
```

```
29          dA = dA * layer.dropMask;
30
31          // Compute dZ = dA * sigma'(Z), unless this is a combined
                gradient
32          Matrix dZ(dA.rows, dA.cols);
33          bool isCombinedGrad =
34              (l == (int)layers_.size() - 1) &&
35              ((lossType == Loss::BinaryCrossEntropy  &&
36                layer.activation == Activation::Sigmoid) ||
37               (lossType == Loss::CategoricalCrossEntropy &&
38                layer.activation == Activation::Softmax));
39
40          if (isCombinedGrad) {
41              dZ = dA;    // gradient already accounts for activation
42          } else {
43              Matrix actGrad = activationGrad(layer.Z, layer.A,
                    layer.activation);
44              dZ = dA * actGrad;  // element-wise: chain rule
45          }
46
47          // Get this layer's input
48          const Matrix& layerInput = (l == 0) ? X : layers_[l-1].A;
49
50          // Compute weight and bias gradients
51          layer.dW = layerInput.transpose().dot(dZ);    // (inFeatures
                x batch).(batch x units)
52          layer.db = dZ.sum(0);                          // sum over
                batch → (1 x units)
53
54          // Optional gradient clipping
55          if (gradClip > 0.0) {
56              auto clipGrad = [&](Matrix& g) {
57                  double norm = std::sqrt(g.square().sum());
58                  if (norm > gradClip) g *= (gradClip / norm);
59              };
60              clipGrad(layer.dW);
61              clipGrad(layer.db);
62          }
63
64          // Propagate error signal to the previous layer
65          if (l > 0) dA = dZ.dot(layer.W.transpose());
66      }
67  }
```

Listing 4.8: MLP.cpp — backward pass (full)

4.6.2 Walking Through Backprop with a Diagram

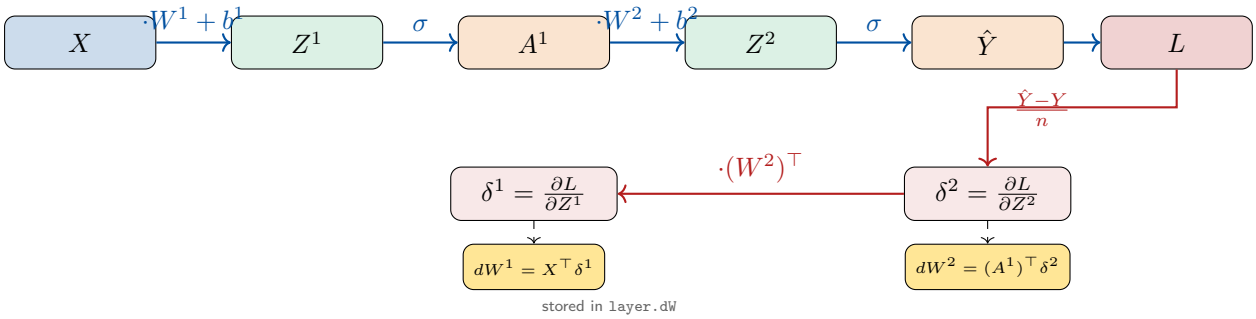


Figure 4.4: Full backpropagation data flow. The error signal $\delta^l = \frac{\partial L}{\partial Z^l}$ flows right to left; weight gradients are computed at each layer.

4.6.3 The Combined Gradient Trick

You might wonder why all three loss functions produce the same gradient formula `(A-Y)/n`. This is a mathematical coincidence for these specific pairings:

- **MSE + linear output**: $\frac{\partial L}{\partial z} = \frac{\hat{y}-y}{n}$ directly.
- **BCE + sigmoid output**: The sigmoid derivative $\sigma'(z) = \sigma(z)(1-\sigma(z))$ cancels with the BCE derivative $\frac{\hat{p}-y}{\hat{p}(1-\hat{p})}$, leaving $\frac{\hat{p}-y}{n}$.
- **CCE + softmax output**: Similarly, the softmax Jacobian collapses with the CCE gradient, leaving $\frac{\hat{p}-y}{n}$.

The code checks for these pairings with `isCombinedGrad` and skips the separate activation gradient multiplication, preventing double-application of the chain rule.

4.6.4 Gradient Clipping

When gradients explode (become very large), training diverges. Gradient clipping rescales the gradient if its L2 norm exceeds a threshold:

```
auto clipGrad = [&](Matrix& g) {
    double norm = std::sqrt(g.square().sum());   // ||g||_2
    if (norm > gradClip) g *= (gradClip / norm); // rescale to unit
        ball
};
```

This preserves the direction of the gradient while bounding its magnitude.

4.7 The SGD Optimizer with Momentum

```cpp
void MLP::updateSGD(double lr, double momentum) {
    for (auto& layer : layers_) {
        // velocity = momentum * velocity - lr * gradient
        layer.velW = layer.velW * momentum - layer.dW * lr;
        layer.velb = layer.velb * momentum - layer.db * lr;
        // weights += velocity
        layer.W += layer.velW;
        layer.b += layer.velb;
    }
}
```

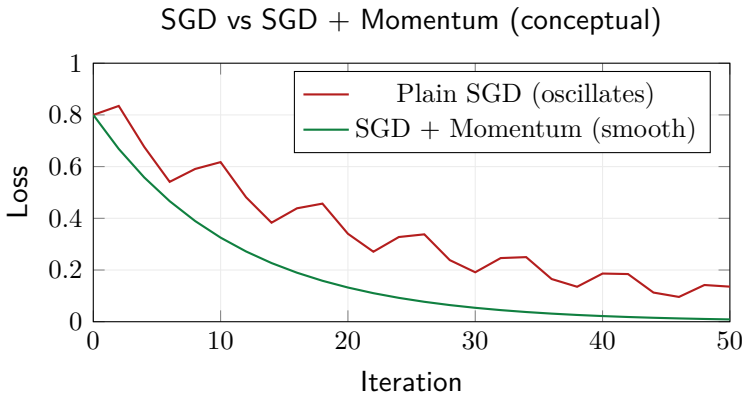
Listing 4.9: MLP.cpp — SGD with momentum

SGD with Momentum

Standard SGD: $W \leftarrow W - \alpha \nabla W$
With momentum (β typically 0.9):

$$v \leftarrow \beta v - \alpha \nabla W$$
$$W \leftarrow W + v$$

Momentum accumulates a velocity vector in directions of persistent gradient descent, accelerating convergence in low-curvature directions and dampening oscillations in high-curvature ones.



4.8 The Adam Optimizer

Adam (Adaptive Moment Estimation) maintains per-parameter learning rates by tracking the first moment (mean of gradients) and second moment (uncentred variance of gradients).

```cpp
void MLP::updateAdam(double lr, double beta1, double beta2, double
    eps) {
    ++adamStep_;   // t = 1, 2, 3, ...

    // Bias correction factors
    double bc1 = 1.0 - std::pow(beta1, adamStep_);  // 1 - beta1^t
    double bc2 = 1.0 - std::pow(beta2, adamStep_);  // 1 - beta2^t
```

```
7
8       for (auto& layer : layers_) {
9           // --- Weights ---
10          // m = beta1 * m + (1 - beta1) * grad      (1st moment: EMA
                of gradients)
11          layer.mW = layer.mW * beta1 + layer.dW * (1.0 - beta1);
12          // v = beta2 * v + (1 - beta2) * grad^2    (2nd moment: EMA
                of squared grads)
13          layer.vW = layer.vW * beta2 + layer.dW.square() * (1.0 -
                beta2);
14
15          // Bias-corrected estimates
16          Matrix mWhat = layer.mW / bc1;    // m_hat
17          Matrix vWhat = layer.vW / bc2;    // v_hat
18
19          // Update: W -= lr * m_hat / (sqrt(v_hat) + epsilon)
20          layer.W -= (mWhat / (vWhat.sqrt() + eps)) * lr;
21
22          // --- Biases (identical procedure) ---
23          layer.mb = layer.mb * beta1 + layer.db * (1.0 - beta1);
24          layer.vb = layer.vb * beta2 + layer.db.square() * (1.0 -
                beta2);
25          Matrix mbhat = layer.mb / bc1;
26          Matrix vbhat = layer.vb / bc2;
27          layer.b -= (mbhat / (vbhat.sqrt() + eps)) * lr;
28      }
29  }
```

Listing 4.10: MLP.cpp — Adam optimizer

Adam Update Rule

Given step t, gradient g_t, defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$:

$$
\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t && \text{(biased 1st moment estimate)} \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 && \text{(biased 2nd moment estimate)} \\
\hat{m}_t &= \frac{m_t}{1-\beta_1^t} && \text{(bias-corrected 1st moment)} \\
\hat{v}_t &= \frac{v_t}{1-\beta_2^t} && \text{(bias-corrected 2nd moment)} \\
W_t &= W_{t-1} - \frac{\alpha}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t && \text{(parameter update)}
\end{aligned}
$$

The $\sqrt{\hat{v}_t}$ in the denominator divides by the *RMS* of recent gradients — parameters that had large gradients get small effective learning rates (they are "well-explored"); parameters with small gradients get large effective learning rates (they need more exploration).

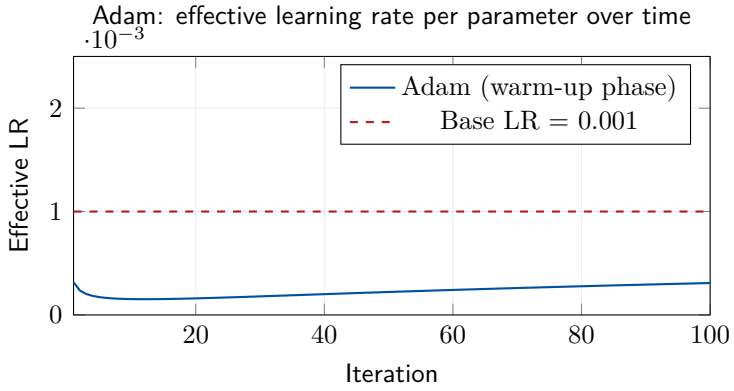


Figure 4.5: Adam's effective learning rate starts lower than the base rate (due to bias correction at small t) and climbs toward the base rate as bias correction terms approach 1.

4.9 The Training Loop: `fit()`

```
void MLP::fit(const Matrix& X, const Matrix& Y, const TrainConfig&
    cfg) {
    // Initialize weights
    for (auto& layer : layers_)
        initWeights(layer, cfg.weightInit);

    adamStep_ = 0;
    lossHistory_.clear();
    int n = X.rows;

    // Create shuffle index
    std::vector<int> idx(n);
    std::iota(idx.begin(), idx.end(), 0);   // fill: 0, 1, 2, ...,
       n-1

    setTraining(true);

    for (int epoch = 0; epoch < cfg.epochs; ++epoch) {
        // Fisher-Yates shuffle for random mini-batches
        for (int i = n - 1; i > 0; --i) {
            int j = std::rand() % (i + 1);
            std::swap(idx[i], idx[j]);
        }

        double epochLoss = 0.0;
        int    numBatches = 0;

        for (int start = 0; start < n; start += cfg.batchSize) {
            int end = std::min(start + cfg.batchSize, n);
            int bs  = end - start;

            // Build mini-batch matrices by gathering shuffled rows
            Matrix Xb(bs, X.cols), Yb(bs, Y.cols);
            for (int i = 0; i < bs; ++i) {
                int row = idx[start + i];
```

```
34                  for (int c = 0; c < X.cols; ++c)
35                      Xb.data[i*X.cols+c] = X.data[row*X.cols+c];
36                  for (int c = 0; c < Y.cols; ++c)
37                      Yb.data[i*Y.cols+c] = Y.data[row*Y.cols+c];
38              }
39
40              // Forward → Loss → Backward → Update
41              Matrix preds = forward(Xb);
42              double batchLoss = computeLoss(preds, Yb, cfg.loss);
43              epochLoss += batchLoss;
44              ++numBatches;
45              backward(Xb, Yb, cfg.loss, cfg.gradClip);
46
47              if (cfg.optimizer == Optimizer::SGD)
48                  updateSGD(cfg.learningRate, cfg.momentum);
49              else
50                  updateAdam(cfg.learningRate, cfg.momentum,
                       cfg.beta2, cfg.epsilon);
51          }
52
53          double avgLoss = epochLoss / numBatches;
54          lossHistory_.push_back(avgLoss);
55
56          if (cfg.verbose && (epoch + 1) % cfg.printEvery == 0) {
57              std::cout << "Epoch [" << std::setw(5) << epoch+1
58                        << "/" << cfg.epochs << "]"
59                        << "  Loss: " << std::fixed <<
                             std::setprecision(6)
60                        << avgLoss << "\n";
61          }
62      }
63
64      setTraining(false);
65  }
```

Listing 4.11: MLP.cpp — training loop

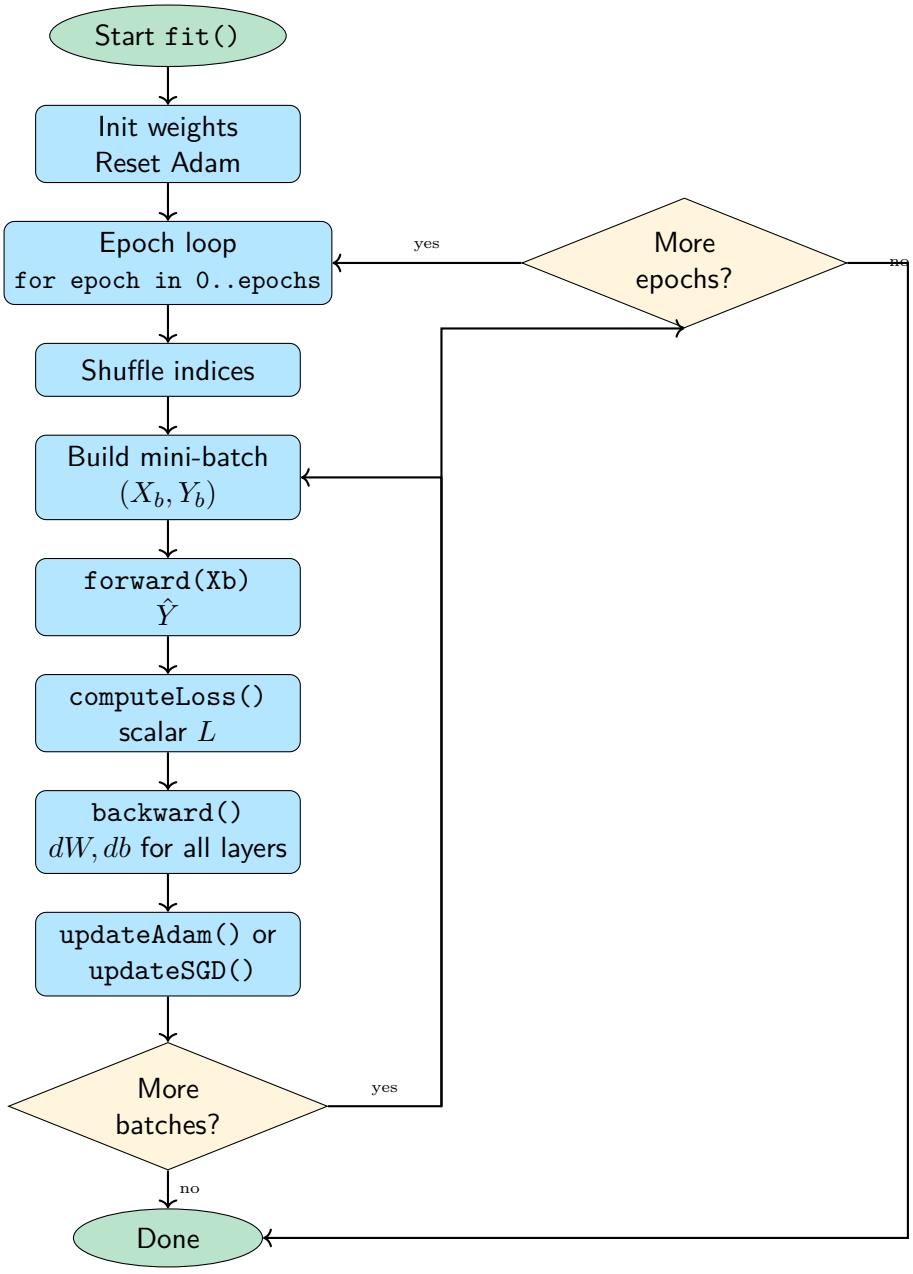


Figure 4.6: Flowchart of the `fit()` training loop. Each epoch shuffles the data, splits into mini-batches, and runs forward + backward + update for each batch.

4.10 Inference and Metrics

```
Matrix MLP::predict(const Matrix& X) {
    setTraining(false);   // disable dropout
    return forward(X);
}

double MLP::accuracy(const Matrix& X, const Matrix& Y) {
    Matrix preds = predict(X);
    int correct = 0, total = X.rows;
```

```
9
10      if (Y.cols == 1) {
11          // Binary: threshold at 0.5
12          for (int r = 0; r < total; ++r) {
13              int pred_class = preds.data[r] >= 0.5 ? 1 : 0;
14              int true_class = Y.data[r] >= 0.5      ? 1 : 0;
15              if (pred_class == true_class) ++correct;
16          }
17      } else {
18          // Multi-class: argmax
19          for (int r = 0; r < total; ++r) {
20              int predIdx = 0, trueIdx = 0;
21              double predMax = preds.data[r*preds.cols];
22              double trueMax = Y.data[r*Y.cols];
23              for (int c = 1; c < preds.cols; ++c) {
24                  if (preds.data[r*preds.cols+c] > predMax)
25                      { predMax = preds.data[r*preds.cols+c]; predIdx
                            = c; }
26                  if (Y.data[r*Y.cols+c] > trueMax)
27                      { trueMax = Y.data[r*Y.cols+c]; trueIdx = c; }
28              }
29              if (predIdx == trueIdx) ++correct;
30          }
31      }
32      return (double)correct / total;
33  }
```

Listing 4.12: MLP.cpp — predict and accuracy

4.11 The Model Summary Printer

```
1   void MLP::printSummary() const {
2       // ... (box drawing characters) ...
3       long long totalParams = 0;
4       for (int l = 0; l < (int)layers_.size(); ++l) {
5           const Layer& layer = layers_[l];
6           long long params = (long long)layer.W.rows * layer.W.cols +
                layer.b.cols;
7           totalParams += params;
8           std::cout << "Layer " << l+1 << ": "
9                     << layer.inFeatures << " → " << layer.units
10                    << "  |  " << actName(layer.activation)
11                    << "  |  params: " << params << "\n";
12      }
13      std::cout << "Total trainable params: " << totalParams << "\n";
14  }
```

Listing 4.13: MLP.cpp — printSummary

The number of parameters in a layer is: inFeatures × units (weights) +units (biases).

4.12 Chapter Summary

What we learned in Chapter 4

- The MLP constructor allocates all weight, gradient, and optimizer matrices at network creation.
- The forward pass: $Z = X \cdot W + b$, then $A = \sigma(Z)$, with dropout applied during training.
- Softmax uses the subtract-max trick for numerical stability.
- The backward pass implements the chain rule: $\delta^l = \delta^{l+1} \cdot (W^{l+1})^\top \odot \sigma'(Z^l)$.
- BCE+Sigmoid and CCE+Softmax have combined gradients $(A - Y)/n$, avoiding double-application of the chain rule.
- SGD momentum accumulates velocity; Adam adapts per-parameter learning rates using first and second moment estimates.
- The training loop shuffles data, builds mini-batches, and runs forward-backward-update for each.

Chapter 5

The Matrix in Action: `main.cpp`

We now have a complete neural network engine. Chapter 5 shows how to use it — building datasets, configuring the network, training, and interpreting the results, through four carefully chosen demo experiments.

5.1 Program Structure

```cpp
#include "MLP.hpp"       // includes Matrix.hpp transitively
#include <cmath>
#include <cstdlib>
#include <ctime>
#include <iostream>
#include <iomanip>

int main() {
    std::srand((unsigned)std::time(nullptr));  // seed random number
        generator

    demoXOR();
    demoCircle();
    demoSineRegression();
    demoMultiClass();

    return 0;
}
```

Listing 5.1: main.cpp — structure overview

`std::srand(std::time(nullptr))` seeds the random number generator with the current time, so each run produces different random weight initialisations and data shuffles.

5.2 Demo 1: XOR — The Classic Test

The XOR function is non-linearly separable: no straight line can divide the class-0 points $\{(0,0),(1,1)\}$ from the class-1 points $\{(0,1),(1,0)\}$. A single-layer network (linear classifier) cannot solve XOR — but a two-layer MLP can.

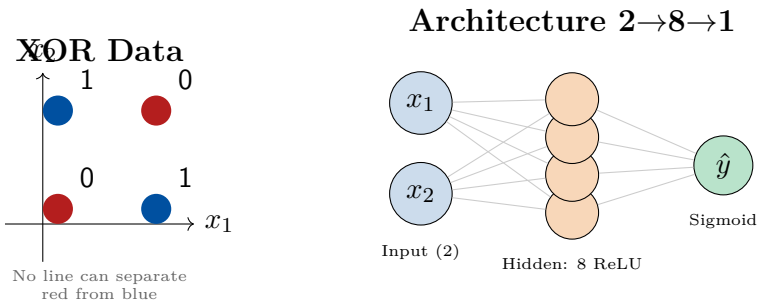


Figure 5.1: XOR data (left) and the network architecture that solves it (right). 8 hidden ReLU neurons create a non-linear decision boundary.

```cpp
const int REPEAT = 50;
const int N      = 4 * REPEAT;   // 200 samples total

Matrix X(N, 2), Y(N, 1);
double xorData[4][2]  = {{0,0},{0,1},{1,0},{1,1}};
double xorLabel[4]    = {0, 1, 1, 0};

for (int rep = 0; rep < REPEAT; ++rep)
    for (int i = 0; i < 4; ++i) {
        int row = rep*4 + i;
        X.at(row, 0) = xorData[i][0];
        X.at(row, 1) = xorData[i][1];
        Y.at(row, 0) = xorLabel[i];
    }
```

Listing 5.2: main.cpp — XOR dataset construction

We repeat the four XOR examples 50 times each because neural networks need enough data for gradient estimation to be stable.

```cpp
MLP net(2, {
    LayerConfig{8,  Activation::ReLU,    0.0},
    LayerConfig{1,  Activation::Sigmoid, 0.0}
});
net.printSummary();

TrainConfig cfg;
cfg.epochs       = 500;
cfg.batchSize    = 32;
cfg.learningRate = 0.01;
cfg.loss         = Loss::BinaryCrossEntropy;
cfg.optimizer    = Optimizer::Adam;
cfg.weightInit   = WeightInit::Xavier;
cfg.verbose      = true;
cfg.printEvery   = 100;

net.fit(X, Y, cfg);
```

Listing 5.3: main.cpp — XOR network and training

Expected Output

The table shows what the trained network predicts for the four canonical XOR inputs. The goal: outputs close to 0 for (0,0) and (1,1), close to 1 for (0,1) and (1,0).

Input (x_1, x_2)	**Expected**	**Predicted**	**Class**
$(0, 0)$	0	≈ 0.01	0 ✓
$(0, 1)$	1	≈ 0.99	1 ✓
$(1, 0)$	1	≈ 0.99	1 ✓
$(1, 1)$	0	≈ 0.02	0 ✓

5.3 Demo 2: Circle Dataset — Non-linear Binary Classification

Points inside radius 0.6 of the origin are class 1; outside are class 0. The boundary is a circle — a curved shape that no linear classifier can capture.

```
const int N = 400;
Matrix X(N, 2), Y(N, 1);

std::srand(42);   // fixed seed for reproducibility
for (int i = 0; i < N; ++i) {
    double x = ((double)std::rand() / RAND_MAX) * 2.0 - 1.0;  // in [-1, 1]
    double y = ((double)std::rand() / RAND_MAX) * 2.0 - 1.0;
    X.at(i, 0) = x;
    X.at(i, 1) = y;
    Y.at(i, 0) = (std::sqrt(x*x + y*y) < 0.6) ? 1.0 : 0.0;  // circle boundary
}
```

Listing 5.4: main.cpp — circle dataset generation

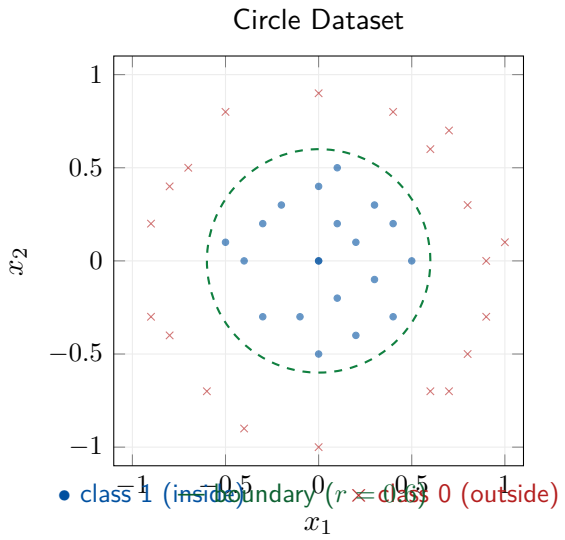


Figure 5.2: Circle dataset: the decision boundary is circular, requiring a non-linear classifier.

```
MLP net(2, {
    LayerConfig{16, Activation::ReLU,    0.0},
    LayerConfig{8,  Activation::ReLU,    0.0},
    LayerConfig{1,  Activation::Sigmoid, 0.0}
});
```

Listing 5.5: main.cpp — circle network

Three layers: two ReLU hidden layers (16 and 8 neurons) and a sigmoid output. This deeper architecture can approximate curved decision boundaries. Expected accuracy: approximately 98%.

Why More Layers?

Each layer applies a non-linear transformation. One hidden layer can approximate any function, but may need exponentially many neurons. Two or three hidden layers learn hierarchical features much more efficiently — the first layer detects simple patterns, the second combines them into complex shapes.

5.4 Demo 3: Sine Regression

Regression (predicting a continuous number rather than a class) uses the MSE loss and a Linear output activation.

```
const double PI = 3.14159265358979;
const int N     = 300;

Matrix X(N, 1), Y(N, 1);
for (int i = 0; i < N; ++i) {
    double x = (double)i / N;
    double noise = (((double)std::rand() / RAND_MAX) - 0.5) * 0.05;
```

```
8       X.at(i, 0) = x;
9       Y.at(i, 0) = std::sin(2.0 * PI * x) + noise;  // sin(2*pi*x) +
            noise
10  }
```

Listing 5.6: main.cpp — sine dataset

```
1   MLP net(1, {
2       LayerConfig{64, Activation::Tanh,   0.0},  // tanh: output in
            (-1,1) like sin
3       LayerConfig{64, Activation::Tanh,   0.0},
4       LayerConfig{1,  Activation::Linear, 0.0}   // no squashing for
            regression
5   });
6
7   TrainConfig cfg;
8   cfg.loss = Loss::MSE;
9   cfg.epochs = 500;
```

Listing 5.7: main.cpp — sine network

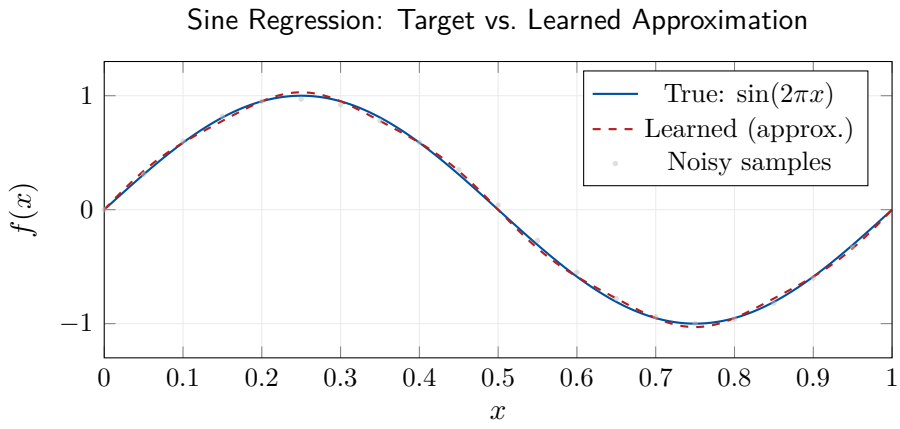


Figure 5.3: After 500 epochs, the MLP closely approximates $\sin(2\pi x)$ from noisy data.

Why Tanh for Sine Regression?

The sine function outputs values in $[-1, 1]$. Tanh also outputs in $(-1, 1)$, so the hidden-layer activations are naturally in the right scale. ReLU would also work but would need more neurons to approximate the negative half of the sine curve (ReLU neurons can only be non-negative).

The **output layer** uses `Linear` (identity) activation — no squashing. Applying sigmoid to the output would clip everything to $(0, 1)$, preventing the network from producing negative values.

5.5 Demo 4: Three-Class Gaussian Blobs

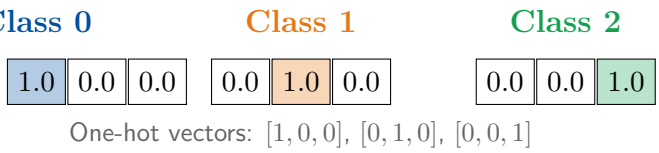
Multi-class classification requires *one-hot encoding* for labels and a *Softmax* output with *Categorical Cross-Entropy* loss.

```cpp
const int N_PER_CLASS = 100;
const int N = 3 * N_PER_CLASS;

double cx[3] = {-1.0, 1.0, 0.0};   // cluster centres x
double cy[3] = { 0.0, 0.0, 1.5};   // cluster centres y

Matrix X(N, 2), Y(N, 3);
Y.fill(0.0);   // initialise one-hot labels to all zeros

for (int cls = 0; cls < 3; ++cls) {
    for (int i = 0; i < N_PER_CLASS; ++i) {
        int row = cls * N_PER_CLASS + i;
        // Generate 2D Gaussian noise around cluster centre
        double angle  = ((double)std::rand() / RAND_MAX) * 6.2832;
           // [0, 2pi]
        double radius = std::sqrt(-2.0*std::log(
                                std::max(1e-9,(double)std::rand()/RAND_MAX)))
                                   * 0.3;
        X.at(row, 0) = cx[cls] + radius * std::cos(angle);
        X.at(row, 1) = cy[cls] + radius * std::sin(angle);
        Y.at(row, cls) = 1.0;   // one-hot: only this class is 1
    }
}
```

Listing 5.8: main.cpp — Gaussian blobs dataset

One-Hot Encoding

For 3-class classification, the label for class 0 is the vector $[1,0,0]^\top$, class 1 is $[0,1,0]^\top$, class 2 is $[0,0,1]^\top$. The output layer has 3 neurons (one per class), and Softmax ensures the outputs sum to 1 (a probability distribution).




```cpp
MLP net(2, {
    LayerConfig{32, Activation::ReLU,    0.0},
    LayerConfig{16, Activation::ReLU,    0.0},
    LayerConfig{3,  Activation::Softmax, 0.0}  // 3 outputs, softmax
});

TrainConfig cfg;
cfg.loss     = Loss::CategoricalCrossEntropy;
cfg.epochs   = 400;
cfg.learningRate = 0.003;
```

Listing 5.9: main.cpp — multi-class network

After training, to predict the class of a new point, we take the argmax of the 3-dimensional softmax output — the index of the highest probability.

5.6 Compiling and Running

```
# On Linux/macOS (GCC or Clang):
g++ -O2 -std=c++17 main.cpp Matrix.cpp MLP.cpp -o synapse
./synapse

# On Windows (with MinGW):
g++ -O2 -std=c++17 main.cpp Matrix.cpp MLP.cpp -o synapse.exe
.\synapse.exe

# Optional: AddressSanitizer to catch memory bugs
g++ -O1 -std=c++17 -fsanitize=address main.cpp Matrix.cpp MLP.cpp -o
    synapse_asan
./synapse_asan
```

Listing 5.10: Compilation and execution

The -O2 Flag

`-O2` enables compiler optimisations: inlining small functions, loop unrolling, and vectorisation. The matrix multiplication loop in particular benefits enormously — training can be 5–10× faster with `-O2` than without. Always use it for ML workloads.

5.7 Reading the Output

A typical training run produces output like:

```
+--------------------------+
|          Synapse.cpp  -  MLP Architecture           |
+--------------------------+
|  Input size :                                  2  |
+--------------------------+
|  Layer 1  :   2 →   8  |        ReLU  |  params:    25  |
|  Layer 2  :   8 →   1  |     Sigmoid  |  params:     9  |
+--------------------------+
|  Total trainable params :                        34  |
+--------------------------+

Epoch [  100/500]  Loss: 0.693142
Epoch [  200/500]  Loss: 0.312891
Epoch [  300/500]  Loss: 0.054203
Epoch [  400/500]  Loss: 0.012318
Epoch [  500/500]  Loss: 0.004102

[XOR] Final training accuracy: 100.00%
```

The loss steadily decreases — the network is learning. If the loss stalls early, try increasing the learning rate or number of epochs. If it diverges (NaN or infinity), decrease the learning rate or enable gradient clipping (`cfg.gradClip = 1.0`).

5.8 Full Data Flow: End to End

Let us trace a single forward-backward-update cycle through the complete system with explicit dimensions for the XOR demo (batch size 32, architecture 2→8→1):

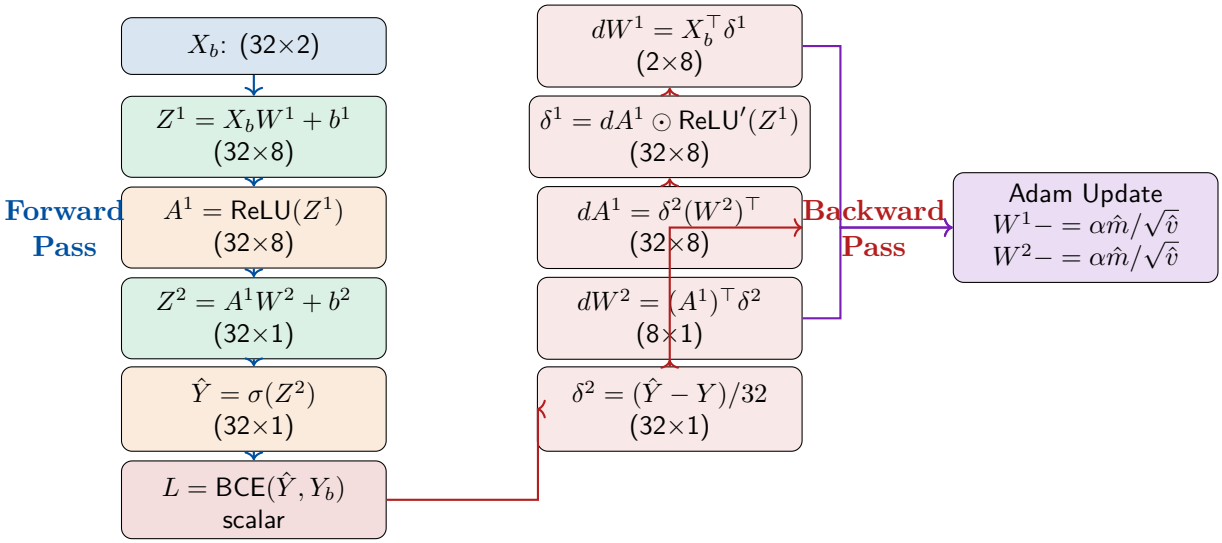


Figure 5.4: One complete training step for the XOR demo: forward pass, loss, backward pass, Adam update. Each matrix has its shape annotated.

5.9 Chapter Summary

What we learned in Chapter 5

- `std::srand(std::time(nullptr))` seeds randomness — essential for reproducible-ish results.
- XOR requires a non-linear hidden layer; BCE loss + Sigmoid output is the standard for binary classification.
- The circle dataset demonstrates that deeper networks learn curved decision boundaries.
- Sine regression uses MSE loss and a Linear output — never use Sigmoid for regression.
- Multi-class classification uses one-hot labels, Softmax output, and Categorical Cross-Entropy.
- The `-O2` compiler flag is essential for acceptable training speed.

Appendix A

Mathematical Quick Reference

A.1 Matrix Shapes in Synapse.cpp

Symbol	Shape	Description
X	(batch $\times$ inFeatures)	Input batch
W^l	(inFeatures$_l$ $\times$ units$_l$)	Layer l weights
b^l	(1 $\times$ units$_l$)	Layer l biases
Z^l	(batch $\times$ units$_l$)	Pre-activation
A^l	(batch $\times$ units$_l$)	Post-activation
$\hat{Y}$	(batch $\times$ outputUnits)	Predictions
dW^l	same as W^l	Weight gradients
δ^l	same as Z^l	Error signal

A.2 Backprop Formulas

$$\begin{aligned}
\text{Output gradient (combined):} \quad & \delta^L = (\hat{Y} - Y)/n \\
\text{Hidden layer error:} \quad & \delta^l = \left(\delta^{l+1} \cdot (W^{l+1})^\top\right) \odot \sigma'(Z^l) \\
\text{Weight gradient:} \quad & \frac{\partial L}{\partial W^l} = (A^{l-1})^\top \cdot \delta^l \\
\text{Bias gradient:} \quad & \frac{\partial L}{\partial b^l} = \sum_{\text{batch}} \delta^l
\end{aligned}$$

A.3 Activation Derivatives

Activation	$\sigma(z)$	$\sigma'(z)$
ReLU	$\max(0, z)$	$\mathbf{1}[z > 0]$
Leaky ReLU	$\max(0.01z, z)$	0.01 or 1
Sigmoid	$\frac{1}{1+e^{-z}}$	$A(1 - A)$
Tanh	$\tanh(z)$	$1 - A^2$
Linear	z	1

Appendix B

Glossary of C++ Terms

`new T[n]` Allocates n objects of type T on the heap. Returns a pointer `T*`.

`delete[] p` Frees the heap array pointed to by `p`.

`nullptr` The null pointer constant (pointer that points to nothing).

`const` Prevents modification. `const Matrix& m` = read-only reference to a Matrix.

`noexcept` Promises a function will not throw exceptions (enables optimisations).

`&` **(reference)** Alias to an existing variable. No copy, no null.

`*` **(dereference)** Get the value at a pointer address.

`#pragma once` Header guard: include this file only once per translation unit.

`std::vector<T>` A dynamically-resizable heap-allocated array from the standard library.

`enum class` A scoped, strongly-typed enumeration.

`auto` Let the compiler deduce the type automatically.

`[[nodiscard]]` Warning if a return value is ignored.

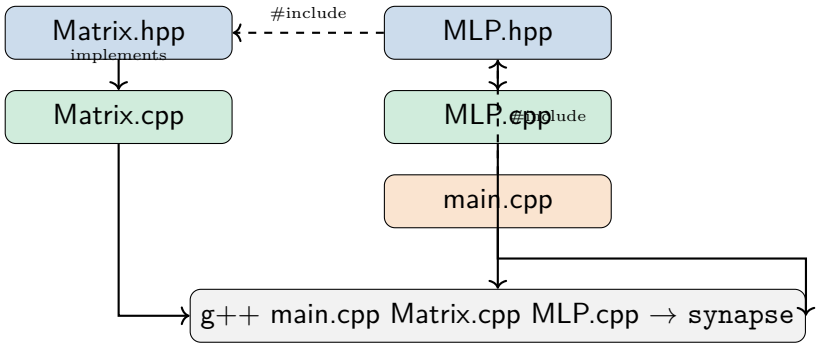
`std::move` Casts to an rvalue reference, enabling move semantics.

Rule of Three If you define destructor, copy constructor, or copy assignment — define all three.

RAII Resource Acquisition Is Initialisation: tie resource lifetime to object lifetime.

Appendix C

Project File Map



Final Note

You have now read a complete, working neural network — every matrix multiplication, every gradient, every byte of heap memory — in raw `C++` without a single external library.

This is not how production ML is written. PyTorch and JAX are faster, safer, and more feature-complete. But having built a neural network from scratch, you now understand what those libraries do internally. When you see `loss.backward()` in PyTorch, you know it is walking a computation graph backwards, multiplying Jacobians exactly as the `backward()` function in Chapter 4 does. When you see `optimizer.step()`, you know it is running the Adam update rule from Section 4.8.

The next frontier: extend Synapse.cpp. Ideas:

- Add batch normalisation (normalise Z^l by its mean and variance across the batch).
- Implement convolutional layers (a sliding-window matrix multiplication).
- Save and load weights to/from binary files.
- Profile the `dot()` function and implement cache-blocking or SIMD vectorisation.
- Write a Python wrapper using `pybind11` so you can use your C++ MLP from Python.

Every great engineer started by reading the source. You are already on your way.

Good luck, and keep building.